

Prim's Algorithm

Topic: Greedy Algorithm — Minimum Spanning Tree (MST)

1. Definition

Prim's Algorithm is a **greedy algorithm** used to find a **Minimum Spanning Tree (MST)** of a connected, weighted and undirected graph. It starts from one vertex and repeatedly adds the minimum-weight edge that connects a visited vertex to an unvisited vertex.

2. Main Idea

- **Start:** Choose any vertex as the starting vertex.
- **Select minimum edge:** Find the smallest edge connecting the current tree to an unvisited vertex.
- **Add vertex:** Add that vertex and edge to the MST.
- **Repeat:** Continue until all vertices are included.
- **Result:** The selected edges form the Minimum Spanning Tree.

3. Student-Style C Program

```
#include <stdio.h>
#define INF 9999

int main()
{
    int n, i, j, u, v;
    int graph[10][10];
    int key[10], parent[10], visited[10];
    int min, total = 0;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter the adjacency matrix:\n");
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            scanf("%d", &graph[i][j]);

            if (graph[i][j] == 0)
                graph[i][j] = INF;
        }
    }

    for (i = 0; i < n; i++)
    {
        key[i] = INF;
        visited[i] = 0;
        parent[i] = -1;
    }

    key[0] = 0;

    for (i = 0; i < n - 1; i++)
    {
        min = INF;
        u = -1;

        for (j = 0; j < n; j++)
        {
            if (!visited[j] && key[j] < min)
            {
                min = key[j];
                u = j;
            }
        }

        visited[u] = 1;
```

```

        for (v = 0; v < n; v++)
        {
            if (!visited[v] && graph[u][v] < key[v])
            {
                key[v] = graph[u][v];
                parent[v] = u;
            }
        }
    }

    printf("\nEdges in Minimum Spanning Tree:\n");

    for (i = 1; i < n; i++)
    {
        printf("%d - %d = %d\n",
               parent[i], i, graph[i][parent[i]]);
        total += graph[i][parent[i]];
    }

    printf("Minimum cost = %d\n", total);

    return 0;
}

```

4. Important Variables

Variable	Purpose
graph[]	Stores the weighted adjacency matrix
key[]	Stores the minimum edge weight needed to connect each vertex
parent[]	Stores the parent of each vertex in the MST
visited[]	Checks whether a vertex is already included
min	Stores the smallest key value found
total	Stores total MST cost

5. Example

Suppose the graph has vertices 0, 1, 2 and 3. Starting from vertex 0, Prim's Algorithm repeatedly chooses the cheapest edge that connects the current MST to a new vertex. After selecting $n-1$ edges, all vertices are connected with minimum total cost.

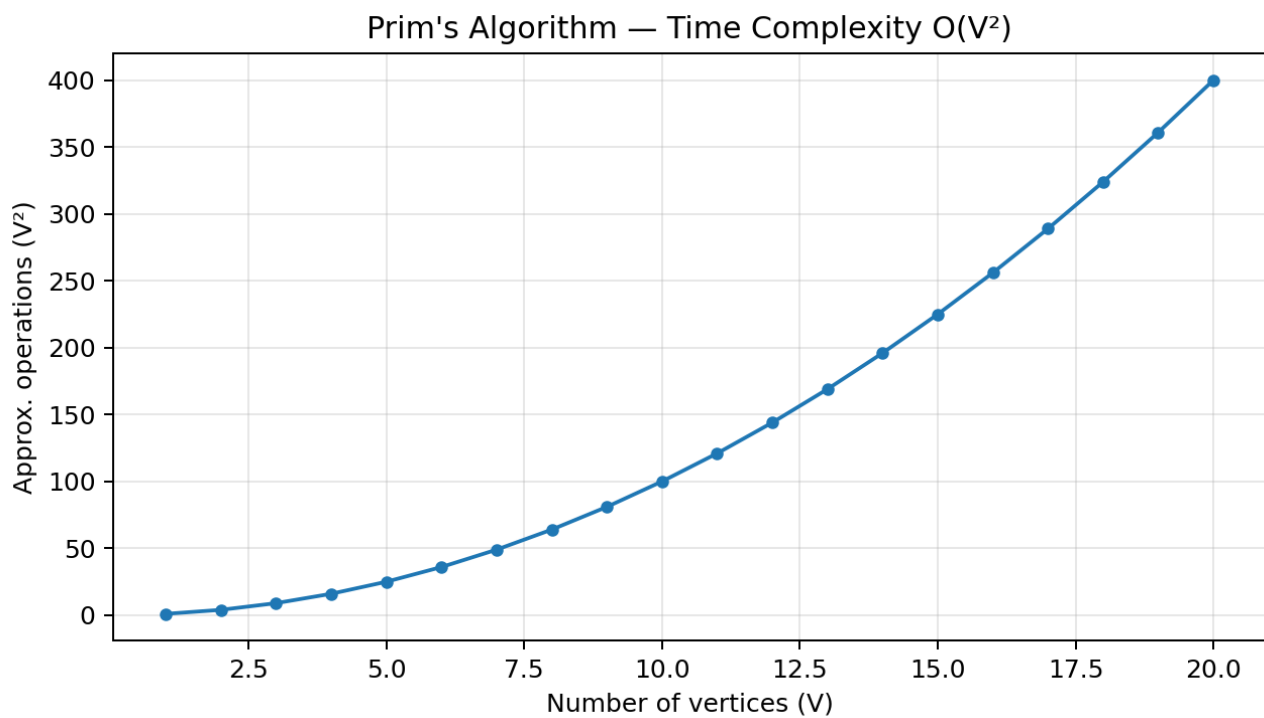
6. Time Complexity

Implementation	Time Complexity	Space Complexity
Adjacency Matrix + Array	$O(V^2)$	$O(V^2)$
Adjacency List + Binary Heap	$O(E \log V)$	$O(V + E)$
Adjacency List + Fibonacci Heap	$O(E + V \log V)$	$O(V + E)$

For This Program: $O(V^2)$

This program uses an **adjacency matrix**. In every iteration, it searches all vertices to find the minimum key, and then scans the vertices again to update their keys. These operations are repeated approximately V times, giving $O(V^2)$ time complexity.

7. Time Complexity Graph



The graph uses V^2 as an operation-growth representation, illustrating the quadratic growth of the standard adjacency-matrix implementation. It is an illustrative complexity graph, not a measurement of actual CPU execution time.

8. Exam Points

- Prim's Algorithm follows the **Greedy approach**.
- It finds a **Minimum Spanning Tree**.
- It works on a connected, weighted, undirected graph.
- An MST of V vertices contains exactly **$V - 1$ edges**.
- For the adjacency-matrix implementation above, time complexity is **$O(V^2)$** .
- The algorithm can start from any vertex.
- Prim's and Kruskal's algorithms both find an MST, but their approaches are different.

9. Prim's vs Kruskal's — Quick Difference

Prim's	Kruskal's
Starts with a vertex	Starts with edges
Grows one tree	Builds a forest and combines components
Chooses minimum edge connected to the current tree	Chooses globally smallest safe edge
Commonly uses priority queue / matrix	Commonly uses sorting + Disjoint Set Union